LocalCloudConfigurator

Interface Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-LocalCloudConfigurator-java. 2 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-LocalCloudConfigurator-go.

LocalCloudConfigurator

LocalCloudConfigurator fills an Arrowhead 5.2.2 local cloud with the support systems and application systems an operator picks. The operator picks the systems in a web UI or through the API. The configurator turns the selection into an ordered deployment plan and submits that plan to [ColonyOS](https://colonyos.io) as a workflow. A ColonyOS executor on the cloud host carries out each step. The first target is the GoCloud (Arrowhead-test-cloud-go). The executor runs the cloud's own scripts, so the configurator follows the same steps as the cloud's Makefile.

Parts

Part	Where	Role
Catalog	catalogs/gocloud.json	The cloud's selectable systems, their Compose services, alternates (Rust/Java swaps), dependencies, identity provisioning and policy scripts
Planner	internal/planner	Turns a selection into steps (provision -> up -> policy, then stop, then smoke) in dependency order
API + UI	localcloudconfigurator serve (container in the ColonyOS stack)	GET /api/catalog, GET /api/status, POST /api/plan, POST /api/deployments, GET /api/deployments/{id}; UI at /
Cloud executor	localcloudconfigurator executor (on the cloud host)	ColonyOS executor of type arrowhead-cloud-executor; serves lcc.status and lcc.step
ColonyOS stack	deploy/colonyos/compose.yaml	TimescaleDB, Colonies server v1.9.10, the API

Rules the executor enforces

- The executor checks every step against its own catalog. It does not trust the arguments of a process.

- Only catalogued Compose services can be started or stopped. Core systems are never touched. - Only catalogued scripts in the cloud's scripts/ directory can run.

- Services start with docker compose up -d --wait --no-deps <service>. Dependencies are explicit plan steps. Compose therefore never starts a Go service while its Rust or Java alternate runs (see the GoCloud swap targets).

- When an alternate runs, the system counts as present. The configurator neither starts nor stops it; the cloud's swap-*/restore-* targets manage it.

- Step output goes to the ColonyOS process logs with credential-like values masked. Secrets are never printed.

- No secret rotation and no PKI changes. Identities are provisioned through the cloud's provision-*-identity.sh scripts, which log in as Sysop with sysop.crt.

LocalCloudConfigurator is an operations tool. It is not an Arrowhead system: it does not register in the ServiceRegistry and has no Arrowhead identity. The new-system checklist (AMQP ACL, sysMonitor policy, Orchestration grants) therefore does not apply.

Build and test

Deploying and the smoke test are described in DEPLOYMENT.md.

Adding a cloud or a system

Add a system as an entry in the catalog: id, kind (infrastructure/support/application), services, and optionally alternates, requires, provision and policies. To support another cloud, write a catalog with its own composeProject and executorType. Then run one executor per cloud host with -catalog and -cloud-dir.

Listening port

serve listens on 127.0.0.1:8095 by default. Override with -listen host:port, SERVER_PORT=N or -server.port N; the port overrides keep the -listen host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Go interface and design differences

The native Go LocalCloudConfigurator is an operator API and embedded UI using net/http and ColonyOS. It is not assumed equivalent to the Java Arrowhead support-system wrapper. internal/api/api.go registers GET /, GET /healthz, GET /api/catalog, GET /api/status, POST /api/plan, POST /api/deployments and GET /api/deployments/{id}. There is no GET /api/deployments collection route in this handler. API calls require Authorization: Bearer <operator-token>, checked in constant time; an empty configured token or mismatch gives 401 with {error:"operator token required"}. This is an operator bearer token, not an Arrowhead IDENTITY-TOKEN. The health endpoint returns text ok. Security headers include X-Content-Type-Options, X-Frame-Options and Content-Security-Policy. TLS termination and network exposure must follow this implementation's deployment configuration; no Java mutual-TLS parity is inferred. Planning rejects unknown JSON fields, limits request input to 64 KiB, returns 400 on decoding errors, 502 on status lookup failure and 422 on planner rejection. A successful plan gives 200. A deployment with no steps gives 200 and an empty id; otherwise successful workflow submission gives 202. Source snapshots of the API, planner, catalog and ColonyOS adapter are included for exact data types and failure behavior.

Detailed implementation evidence

0 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

No Rust implementation folder for LocalCloudConfigurator was found in the designated primary Rust tree on 2026-11-09. No executable, build command, endpoint or feature parity is asserted. A future port must define architecture, state migration, service registration, DTO validation, authorization and transport coverage before it can replace this system.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-LocalCloudConfigurator-java/documentation/v5.2.2/LocalCloudConfigurator-v5.2.2-localCloudConfiguration-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-LocalCloudConfigurator-java/documentation/v5.2.2/LocalCloudConfigurator-v5.2.2-sysMonitor-AMQP-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

localCloudConfiguration HTTP/TLS/JSON

Interface Design Description

Service ID: *"localCloudConfiguration"*

Abstract
This document describes the HTTP/TLS/JSON interface of the `localCloudConfiguration` service, version 1.0.0, of the LocalCloudConfigurator support system of Arrowhead 5.2.2.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation catalog . . . 4
2.2 operation status . . . 4
2.3 operation plan . . . 5
2.4 operation submit-deployment . . . 5
2.5 operation get-deployment . . . 6
2.6 operation list-configurations . . . 7
2.7 operation create-configuration . . . 7
2.8 operation get-configuration . . . 8
2.9 operation update-configuration . . . 8
2.10 operation remove-configuration . . . 8

3 Data Models **10**
3.1 struct ApplyRequest . . . 10
3.2 struct Catalog . . . 10
3.3 struct Component . . . 10
3.4 struct Configuration . . . 10
3.5 struct ConfigurationDraft . . . 11
3.6 struct ConfigurationList . . . 11
3.7 struct Deployment . . . 11
3.8 struct DeploymentAccepted . . . 11
3.9 struct Plan . . . 11
3.10 struct PlanRequest . . . 12
3.11 struct PlanResponse . . . 12
3.12 struct Point . . . 12
3.13 struct Removal . . . 12
3.14 struct RevisionCheck . . . 12
3.15 struct Status . . . 12
3.16 struct Step . . . 12
3.17 struct StepState . . . 13
3.18 Primitives . . . 13

4 References **14**

5 Revision History **15**
5.1 Amendments . . . 15
5.2 Quality Assurance . . . 15

1 Overview

This document describes the `localCloudConfiguration` service interface of the LocalCloudConfigurator system.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `localCloudConfiguration` service interface

This document provides the Interface Design Description IDD to the *localCloudConfiguration – Service Description* document [5].

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, port 8492, base path `/api` and the service metadata `systemVersion=5.2.2`. Path parameters are templates (`{id}`):

Operation	Method	Path
`catalog`	GET	`/{}catalog`
`status`	GET	`/{}status`
`plan`	POST	`/{}plan`
`submit-deployment`	POST	`/{}deployments`
`get-deployment`	GET	`/{}deployments/{}{id}`
`list-configurations`	GET	`/{}configurations`
`create-configuration`	POST	`/{}configurations`
`get-configuration`	GET	`/{}configurations/{}{id}`
`update-configuration`	PUT	`/{}configurations/{}{id}`
`remove-configuration`	DELETE	`/{}configurations/{}{id}`

2 Service Operations

Each subsection names the HTTP method and path of an operation, the abstract operation of the SD and its input and output types.

Every request needs the caller's client certificate (mutual TLS) and `Authorization: Bearer IDENTITY-TOKEN//<token>` of the same system; the operation is allowed for Sysop, the systems of `lcc.api.whitelist` and systems with a ConsumerAuthorization grant of the operation. These checks answer 401 and 403 with the common error body (`errorMessage`, `errorCode`, `exceptionType`, `origin`); other `/api` paths are refused (403) before any handler. Request bodies are JSON of at most 64 KiB (413 `"Request exceeds 64 KiB"`); unknown fields are rejected (400). Errors of the operations are `{"error": "<text>"}` with 400 (invalid input), 404, 409 (stale plan, running deployment, changed design), 413 and 502 (backend failure, redacted message). Responses carry `Cache-Control: no-store`, `X-Content-Type-Options: nosniff` and `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'`. Absent `null` fields are omitted.

2.1 GET /{}api/{}catalog

Operation: catalog
Output: Catalog

The component catalog of the cloud.

A successful call returns `200 OK`.

```
GET /api/catalog HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
```

Listing 1: An catalog invocation.

```
{
  "cloud": "TestCloud",
  "composeProject": "arrowhead-v52-test-cloud",
  "colony": "arrowhead-java-localcloud",
  "executorType": "arrowhead-java-cloud-executor",
  "defaults": ["serviceregistry", "authentication",
               "consumerauthorization", "serviceorchestration-dynamic"],
  "smoke": ["verify-localcloud"],
  "systems": [{"id": "timemanager", "name": "TimeManager", "kind": "support",
               "services": ["timemanager-db", "timemanager"],
               "internalAssets": ["timemanager-db"],
               "requires": ["amqp-broker", "consumerauthorization"],
               "provision": ["provision-timemanager-identity.sh"],
               "policies": ["configure-timemanager-policy.sh"],
               "protected": false}, ...]
}
```

Listing 2: An catalog response.

2.2 GET /{}api/{}status

Operation: status
Output: Status

Running Compose services and the state (`running`, `partial`, `stopped`) of each component (cached 2 s).

A successful call returns `200 OK`. Errors: 502 when the backend cannot report the runtime.

```
GET /api/status HTTP/1.1
Host: localcloudconfigurator:8492
```

```
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
```

Listing 3: An status invocation.

```
{
  "cloud": "TestCloud",
  "running": ["serviceregistry-db", "serviceregistry", ...],
  "systems": [{"id": "serviceregistry", "state": "running"},
              {"id": "timemanager", "state": "stopped"}, ...]
}
```

Listing 4: An status response.

2.3 POST /{}api/{}plan

Operation: plan
Input: PlanRequest
Output: PlanResponse

Builds a plan against the current runtime; returns it with its hash.

A successful call returns `200 OK`. Errors: 400 for an unknown component, a forbidden removal (protected, selected, still required) or a dependency cycle.

```
POST /api/plan HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
Content-Type: application/json

{
  "systems": ["timemanager"],
  "remove": [],
  "reapply": false,
  "smoke": true
}
```

Listing 5: An plan invocation.

```
{
  "plan": {
    "cloud": "TestCloud",
    "catalogHash": "9f2c...",
    "systems": ["amqp-broker", "serviceregistry", ..., "timemanager"],
    "steps": [{"kind": "provision", "target": "provision-timemanager-identity.sh",
               "system": "timemanager"},
              {"kind": "up", "target": "timemanager-db", "system": "timemanager"},
              {"kind": "up", "target": "timemanager", "system": "timemanager"},
              {"kind": "policy", "target": "configure-timemanager-policy.sh",
               "system": "timemanager"},
              {"kind": "smoke", "target": "verify-localcloud"}],
    "notes": ["ServiceRegistry: already running", ...]
  },
  "hash": "41d8..."
}
```

Listing 6: An plan response.

2.4 POST /{}api/{}deployments

Operation: submit-deployment
Input: ApplyRequest
Output: DeploymentAccepted

Applies a reviewed plan if its hash still matches a fresh plan and no other deployment is running.

A successful call returns `202 Accepted` (`200 OK` when the plan has no steps). Errors: 409 `"Plan or cloud state changed. Review a fresh plan before applying."` or `"A deployment is still running. Wait for it before applying another plan."`; 400 as for plan; 502 when the backend fails.

```
POST /api/deployments HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
Content-Type: application/json

{
  "systems": ["timemanager"],
  "remove": [],
  "reapply": false,
  "smoke": true,
  "expectedPlanHash": "41d8..."
}
```

Listing 7: An submit-deployment invocation.

```
{
  "id": "local-0f8e2a1c-6d4b-4a3e-9c1d-2b7f5e8a9c01",
  "plan": { ... },
  "state": "submitted"
}
```

Listing 8: An submit-deployment response.

2.5 GET /{}api/{}deployments/{}*{id}*

Operation: get-deployment
Input: Identifier
Output: Deployment

State and step results of a submitted deployment (identifier appended to the path); the report is stored.

A successful call returns `200 OK`. Errors: 404 `"Deployment not found"` for an identifier this system did not submit (or a historical ColonyOS workflow with the direct backend).

```
GET /api/deployments/local-0f8e2a1c-6d4b-4a3e-9c1d-2b7f5e8a9c01 HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
```

Listing 9: An get-deployment invocation.

```
{
  "id": "local-0f8e2a1c-6d4b-4a3e-9c1d-2b7f5e8a9c01",
  "state": "running",
  "steps": [{"node": "s1", "processId": "", "kind": "up",
             "target": "timemanager-db", "state": "success",
             "errors": [], "output": "..."},
            {"node": "s2", "processId": "", "kind": "up",
             "target": "timemanager", "state": "running",
             "errors": [], "output": ""}]
```

```
}
```

Listing 10: An get-deployment response.

2.6 GET /{}api/{}configurations

Operation: list-configurations
Output: ConfigurationList

Saved designs (up to 1000), sorted by name.
A successful call returns `200 OK`.

```
GET /api/configurations HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
```

Listing 11: An list-configurations invocation.

```
{
  "entries": [{"id": "3f1c9a7e-2b4d-4e6f-8a1b-9c0d2e3f4a5b", "name": "Line 1", ...}]
}
```

Listing 12: An list-configurations response.

2.7 POST /{}api/{}configurations

Operation: create-configuration
Input: ConfigurationDraft
Output: Configuration

Saves a new design (revision 1).
A successful call returns `201 Created`. Errors: 400 for an invalid name, components or positions.

```
POST /api/configurations HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
Content-Type: application/json

{
  "name": "Line 1",
  "systems": ["timemanager"],
  "remove": [],
  "layout": {"timemanager": {"x": 320, "y": 140}}
}
```

Listing 13: An create-configuration invocation.

```
{
  "id": "3f1c9a7e-2b4d-4e6f-8a1b-9c0d2e3f4a5b",
  "name": "Line 1",
  "systems": ["serviceregistry", "authentication", "consumerauthorization",
              "serviceorchestration-dynamic", "timemanager"],
  "remove": [],
  "reapply": false,
  "smoke": false,
  "layout": {"timemanager": {"x": 320.0, "y": 140.0}},
  "revision": 1,
  "updatedAt": "2026-11-09T10:00:00.123456Z"
}
```

Listing 14: An create-configuration response.

2.8 GET /{}api/{}configurations/{}*{id}*

Operation: get-configuration
Input: Identifier
Output: Configuration

One saved design.

A successful call returns `200 OK`. Errors: 404 `"Configuration not found"`.

```
GET /api/configurations/3f1c9a7e-2b4d-4e6f-8a1b-9c0d2e3f4a5b HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
```

Listing 15: An get-configuration invocation.

2.9 PUT /{}api/{}configurations/{}*{id}*

Operation: update-configuration
Input: ConfigurationDraft
Output: Configuration

Replaces a design when `expectedRevision` matches; the revision is incremented.

A successful call returns `200 OK`. Errors: 409 `"Configuration changed; reload before saving"`; 404; 400.

```
PUT /api/configurations/3f1c9a7e-2b4d-4e6f-8a1b-9c0d2e3f4a5b HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
Content-Type: application/json

{
  "name": "Line 1 (edited)",
  "systems": ["timemanager"],
  "expectedRevision": 1
}
```

Listing 16: An update-configuration invocation.

2.10 DELETE /{}api/{}configurations/{}*{id}*

Operation: remove-configuration
Input: RevisionCheck
Output: Removal

Deletes a design when `expectedRevision` matches; the runtime is unchanged.

A successful call returns `200 OK`. Errors: 409 `"Configuration changed; reload before deleting"`; 404.

```
DELETE /api/configurations/3f1c9a7e-2b4d-4e6f-8a1b-9c0d2e3f4a5b HTTP/1.1
Host: localcloudconfigurator:8492
Authorization: Bearer IDENTITY-TOKEN//<token of LocalCloudManager>
Content-Type: application/json

{
```

```
  "expectedRevision": 2
}
```

Listing 17: An remove-configuration invocation.

```
{
  "removed": "3f1c9a7e-2b4d-4e6f-8a1b-9c0d2e3f4a5b"
}
```

Listing 18: An remove-configuration response.

3 Data Models

The JSON objects of the interface, in alphabetic order.

3.1 struct ApplyRequest

Field	Type	Description
systems	List<Name>	Selected components.
remove	List<Name>	Components to stop.
reapply	Boolean	Rerun setup of running components.
smoke	Boolean	Add the verification step.
expectedPlanHash	String	Hash of the reviewed plan (409 if it no longer matches).

3.2 struct Catalog

Field	Type	Description
cloud	Name	Local cloud.
composeProject	Name	Compose project.
colony	Name	ColonyOS colony (legacy backend).
executorType	Name	ColonyOS executor type.
defaults	List<Name>	Default core components.
smoke	List<Name>	Verification operations (`verify-localcloud`).
systems	List<Component>	Components.

3.3 struct Component

Field	Type	Description
id	Name	Component identifier.
name	String	Display name.
kind	String	`core`, `support`, `application` or `infrastructure`.
services	List<Name>	Compose services, databases first.
internalAssets	List<Name>	Owned databases.
requires	List<Name>	Dependencies.
provision	List<String>	Provisioning scripts.
policies	List<String>	Policy scripts.
protected	Boolean	Cannot be stopped.

3.4 struct Configuration

Field	Type	Description
id	Identifier	UUID.
name	String	1–200 characters.
systems	List<Name>	Defaults first, then the selection (databases mapped to their host).
remove	List<Name>	Components marked for stopping.
reapply	Boolean	
smoke	Boolean	
layout	Object<Point>	Positions by component.
revision	Number	Incremented on every update.
updatedAt	DateTime	Time of the last change.

3.5 struct ConfigurationDraft

Field	Type	Description
name	String	1–200 characters.
systems	List<Name>	
remove	List<Name>	
reapply	Boolean	
smoke	Boolean	
layout	Object<Point>	At most 200 positions.
expectedRevision	Number	Required for update-configuration.

3.6 struct ConfigurationList

Field	Type	Description
entries	List<Configuration>	Saved designs.

3.7 struct Deployment

Field	Type	Description
id	Identifier	Deployment identifier.
state	String	`running`, `success` or `failed` (ColonyOS: also `waiting`).
steps	List<StepState>	One entry per plan step.

3.8 struct DeploymentAccepted

Field	Type	Description
id	Identifier	Deployment identifier (empty when there was nothing to do).
plan	Plan	The applied plan.
state	String	`submitted`, or `success` when the plan had no steps.

3.9 struct Plan

Field	Type	Description
cloud	Name	Local cloud.
catalogHash	String	SHA-256 of the catalog.
systems	List<Name>	Components in dependency order.
steps	List<Step>	Ordered steps.
notes	List<String>	For example `"TimeManager: already running"`.

3.10 struct PlanRequest

Field	Type	Description
systems	List<Name>	Selected components (at most 200).
remove	List<Name>	Components to stop (at most 200).
reapply	Boolean	
smoke	Boolean	

3.11 struct PlanResponse

Field	Type	Description
plan	Plan	The plan.
hash	String	SHA-256 of the plan.

3.12 struct Point

Field	Type	Description
x	Number	0–10000.
y	Number	0–10000.

3.13 struct Removal

Field	Type	Description
removed	Identifier	Removed design.

3.14 struct RevisionCheck

Field	Type	Description
expectedRevision	Number	Current revision.

3.15 struct Status

Field	Type	Description
cloud	Name	Local cloud.
running	List<Name>	Running Compose services.
systems	List<Object>	`id` and `state` per component.

3.16 struct Step

Field	Type	Description
kind	String	`provision`, `up`, `policy`, `stop` or `smoke`.
target	String	Script, Compose service or verification.
system	Name	Owning component (absent for `smoke`).

3.17 struct StepState

Field	Type	Description
node	String	Step identifier (`s1`, ...).
processId	String	ColonyOS process (empty for the worker).
kind	String	Step kind.
target	String	Step target.
state	String	`waiting`, `running`, `success` or `failed`.
errors	List<String>	Redacted errors.
output	String	Redacted, bounded command output.

3.18 Primitives

Type	Description
Boolean	JSON Boolean.
DateTime	JSON String, ISO 8601 UTC timestamp (`Instant.toString`).
Identifier	JSON String.
List	JSON Array [4].
Name	JSON String.
Number	JSON Number.
Object	JSON Object.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “localCloudConfiguration – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudConfigurator-v5.2.2-localCloudConfiguration-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-09	5.2.2	Document created for the `localCloudConfiguration` HTTP/TLS/JSON interface of the Java-Spring LocalCloudConfigurator (port 8492).	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the LocalCloudConfigurator system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration 3

2 Service Operations **4**
2.1 operation echo 4
2.2 operation power 4
2.3 operation status 4

3 Data Models **6**
3.1 struct Observation 6
3.2 Primitives 6

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of LocalCloudConfigurator.
It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, the common sysMonitor publisher of LocalCloudConfigurator registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.LocalCloudConfigurator.LocalCloudConfigurator",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher is the `SysMonitorPublisher` of the vendored Eclipse Arrowhead common library (RabbitMQ Java client), configured with the `sysmonitor.*` Spring properties (`enabled`, `broker.address`, `broker.port`, `local.cloud`, `service.instance`, `publish.interval`, `ssl.*`). The echo reports the default health of the common publisher; `serviceCount` is the number of services in the system information (2: `generalManagement` and `localCloudConfiguration`). The publisher connects to the broker (default `amqp-broker`, port 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.LocalCloudConfigurator.LocalCloudConfigurator.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.LocalCloudConfigurator.LocalCloudConfigurator.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.LocalCloudConfigurator.LocalCloudConfigurator.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "LocalCloudConfigurator-echo-1792670400000-42",
  "providerSystem": "LocalCloudConfigurator",
  "serviceInstance": "LocalCloudConfigurator",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "LocalCloudConfigurator is running"
  },
  "quality": "good",
```

```
15    "metadata": {
16      "publisherMode": "native"
17    }
18  }
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power`, `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudConfigurator-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-09	5.2.2	Document created for the sysMonitor service of the Java-Spring LocalCloudConfigurator (common-library publisher).	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	